ATUALIZAÇÃO DA FUNCIONALIDADE • 2026

Serviços bancários e movimentações

Complemento à proposta arquitetural — visão atual da aplicação, seus fluxos e responsabilidades.

Escopo documentado
Clientes e contas, extrato, depósitos, saques, crédito, cartões, pagamentos, benefícios, responsáveis e prestação de contas.

1. Organização em camadas

O sistema é um monólito modular cliente-servidor: uma aplicação Express organiza módulos por responsabilidade; o navegador conversa com a API por HTTP/JSON, e o PostgreSQL é acessado pelo adaptador de infraestrutura.

CLIENTE public/	PRESENTATION controllers	APPLICATION services / casos de uso	DOMAIN regras de conta	INFRASTRUCTURE adaptador PostgreSQL	DADOS PostgreSQL

Direção da requisição: navegador → HTTP/JSON → controller → service → domínio/porta → adaptador PostgreSQL → dados. A resposta retorna pelo caminho inverso.

2. Exemplo de fluxo: pagamento

Etapa	Responsabilidade
Cliente / Controller	A interface envia POST /payments com accountId, payee, description e amount; o controller encaminha ao caso de uso.
Application	CommunityService valida os campos e solicita a movimentação de débito ao AccountService.
Domain	Account valida valor, conta ativa e saldo disponível; calcula saldo em centavos.
Repository / dados	O adaptador bloqueia a conta e persiste saldo, movimento e pagamento na mesma transação PostgreSQL.
Resposta	Sucesso retorna HTTP 201 e atualiza saldo/lista. Falhas não deixam débito parcial.

3. Funcionalidades e contratos HTTP

Controladores adaptam requisições HTTP, serviços coordenam casos de uso e validações, e o adaptador persiste os registros.

Funcionalidade	Rotas principais	Comportamento implementado
Clientes e contas	POST/GET /customers POST/GET /accounts	Cadastra clientes; abre conta ligada a cliente existente ou titular novo.
Extrato e movimentos	GET /accounts/:accountId/statement GET /accounts/:accountId/transactions POST /accounts/:accountId/deposits POST /accounts/:accountId/withdrawals POST /accounts/:accountId/transactions	Consulta histórico, registra depósito/saque; rota genérica aceita type deposit ou withdrawal.
Crédito solidário/pessoal	POST /credit/solidarity POST /credit/personal GET /credit/requests?customerId=...	Pedido com juros 0% e status pending. Solidariedade exige enterpriseName; não aprova nem libera crédito.
Cartão popular	POST /cards GET /cards?accountId=...	Pedido associado à conta, status requested; sem emissão ou integração com operadora.
Pagamento de contas	POST /payments GET /accounts/:accountId/payments	Registra pagamento e debita imediatamente a conta, atomicamente com saldo e movimento.
Benefício	POST /accounts/:accountId/benefits GET /accounts/:accountId/benefits	Registra crédito informado por operador; referência opcional e não repetível na conta.
Responsáveis	GET/POST /accounts/:accountId/managers DELETE /accounts/:accountId/managers/:managerId	Associa, lista ou remove responsáveis/gerentes.
Prestação de contas	GET /reports/daily GET /reports/weekly	Totais por categoria, líquido movimentado, quantidade de movimentos e contas ativas.

4. Dependências entre elementos

Elemento	Responsabilidade e dependência
public/ (cliente)	Apresenta formulários, listas e relatórios; depende de HTTP/JSON, não acessa o banco.
Controllers	Mapeiam rotas/status HTTP e dependem dos serviços; não calculam saldo nem executam SQL.
AccountService / CommunityService	Coordenam casos de uso; dependem de contratos de repositório e domínio. CommunityService reutiliza AccountService para pagamentos/benefícios.
Account (domínio)	Mantém invariantes monetárias e da conta ativa; independente de Express e PostgreSQL.
Repository / PostgreSQL	A porta define persistência; PostgresAccountRepository implementa o contrato e acessa o banco.

5. Regras e integridade financeira

Valores são convertidos para centavos inteiros. A operação financeira acontece em transação PostgreSQL: SELECT ... FOR UPDATE bloqueia a conta, o domínio valida e calcula o saldo, e saldo e lançamento são persistidos juntos.

Condição	Resultado
Tipo ou valor inválido	HTTP 400; nenhuma alteração financeira.
Conta inexistente	HTTP 404; nenhum lançamento.
Conta inativa ou saldo insuficiente	HTTP 409; transação revertida.
Depósito / benefício válido	Crédito ao saldo e lançamento confirmados juntos.
Saque / pagamento válido	Débito limitado ao saldo; saldo e movimento confirmados juntos.

6. Fluxos dos demais serviços

Serviço	Caminho	Persistência / limite
Depósito, saque e extrato	AccountController → AccountService → Account.applyMovement → repositório.	accounts e account_movements; saque não excede saldo; extrato consulta histórico.
Pagamento	CommunityController → CommunityService → AccountService → domínio → repositório.	payments, movimento e saldo atômicos; sem liquidação externa.
Benefício	CommunityController → CommunityService → AccountService → domínio → repositório.	benefit_receipts, movimento e saldo atômicos; lançamento manual.
Solicitação de crédito	CommunityController → CommunityService → repositório.	Pedido pending com juros 0%; não altera saldo ou libera recursos.
Pedido de cartão	CommunityController → CommunityService → repositório.	Pedido requested; não emite cartão nem integra emissor.
Responsável	CommunityController → CommunityService → repositório.	Associação independente das movimentações financeiras.
Relatório	CommunityController → CommunityService → consulta agregada no repositório.	Últimas 24h ou últimos 7 dias, em UTC conforme PostgreSQL.

Limites da versão acadêmica: sem autenticação/autorização. Benefícios são lançados manualmente; pagamentos não se conectam a bancos/concessionárias; crédito e cartões não são aprovados ou emitidos automaticamente.

7. Relação com a proposta arquitetural

A implementação mantém o estilo cliente-servidor e o monólito modular propostos. A separação por camadas restringe dependências: apresentação conhece aplicação; aplicação depende de portas e domínio; infraestrutura adapta as portas ao PostgreSQL. O domínio não depende de detalhes externos.

Os casos de uso financeiros aplicam regras de domínio e consistência ACID. Pagamento e benefício compartilham a movimentação de conta sem duplicar regra de saldo. Crédito, cartão, responsáveis e relatórios são casos de uso próprios — não são apresentados como liberação de recursos que ainda não existe.

8. Arquivos que materializam os fluxos

Parte	Arquivos	Papel
Cliente web	public/index.html public/app.js public/styles.css	Formulários e visualizações; comunicação HTTP/JSON.
Apresentação	src/presentation/account-controller.ts src/presentation/community-controller.ts	Rotas HTTP de clientes, contas e serviços comunitários.
Aplicação e contratos	src/application/account-service.ts src/application/community-service.ts src/application/account-repository.ts	Casos de uso, validações, coordenação e portas.
Domínio	src/domain/account.ts	Movimentos, valores em centavos e invariantes de saldo/conta.
Infraestrutura	src/infrastructure/postgres-account-repository.ts src/app.ts src/server.ts	Persistência PostgreSQL, composição de dependências e inicialização.
Testes	src/domain/account.test.ts src/application/community-service.test.ts	Regras de conta e casos de uso comunitários.

Conclusão

O sistema explicita quem recebe cada requisição, quem coordena o caso de uso, onde ficam as regras e qual adaptador persiste os dados. As integrações externas ainda não implementadas são indicadas como limites do protótipo.

Documento atualizado em 22 de outubro de 2026; conteúdo conferido com os endpoints, camadas e README atuais.